PC-FNO3D — notions du projet et rôle de chaque bloc

Oct 2, 2026 · @Ines

PC-FNO3D reconstruit en 91 ms le vent 3D (25 à 1000 m) sur 50 × 25 km à partir des seuls capteurs au sol, avec un R^2 de u et v entre 0,991 et 0,998.

Le projet en bref

- **Problème** : les capteurs ANEO mesurent le vent seulement au sol ; or les éoliennes, les drones et la pollution dépendent du vent en altitude.
- **Entrée** : les mesures des capteurs au sol (vent, température, densité, humidité, pression) + le relief + l'heure.
- **Sortie** : u, v, w en chaque point d'une grille 3D de 12 niveaux × 250 × 500 cases de 100 m.
- **Méthode** : un réseau de neurones FNO 3D (453 M paramètres) entraîné avec une loss qui impose aussi la physique (PC = *physics-constrained*).
- **Résultats test** : RMSE global u = 0,542 m/s, v = 0,437 m/s ; R^2 de w ⩾ 0,966 ; résidu de masse 2,58e-5 ; rugosité 0,951 ; 0,646 m/s avec 25 % des capteurs ; erreur de direction 3,6°.
- **Déploiement** : application Streamlit + Plotly (Colab + cloudflared, ou Hugging Face Spaces), exports NetCDF / CSV.

Notions : données et grille

Notion	Explication simple	Où dans le code
u, v, w	vent vers l'est, vers le nord, vers le haut (m/s)	partout
T, ρ, RH, p	température, densité de l'air, humidité, pression	`SURF_VARS` (BLOC 3)
Instant	une « photo » du vent à une date et une heure	`stem`, ex. `004_120000` = jour 4, 12 h

Notion	Explication simple	Où dans le code
Grille de surface	cases de 50 m (1000 × 500) où sont les capteurs	`NX_S`, `NY_S`
Grille du modèle	cases de 100 m (500 × 250) × 12 niveaux	`NX`, `NY`, `NZ`, `STRIDE = 2`
Niveaux ASL	altitudes au-dessus de la mer, de 25 à 1000 m	`Z_LEVELS`
ASL / AGL	altitude au-dessus de la mer / hauteur au-dessus du sol	`z_asl`, `z_agl`
Nœud valide	point 3D dans l'air (pas sous le relief)	`VALID`
Relief, rugosité, pente, orientation	forme et nature du sol, qui freinent et dévient le vent	`zt`, `z0`, `slope`, `aspect`
Pooling 2 × 2	moyenne des capteurs de 4 petites cases → 1 grande case (moins de bruit)	`pool_surface`
Cache	données CSV converties une fois en fichiers `.npy` rapides	BLOC 3
Tenseur	tableau de nombres à plusieurs dimensions (ex. 25 × 12 × 250 × 500)	partout
Canal	une couche d'information dans le tenseur (une carte de température…)	`C_IN = 25`
25 canaux d'entrée	7 mesures + 1 carte capteurs + 6 relief + 3 géométrie + 8 temps/physique	`assemble_input`
train / val / test	jours pour apprendre / pour choisir le meilleur modèle / pour la note finale	`SPLIT`, `INDEX["split"]`
Synoptique	instants à heure ronde toutes les 3 h (0 h, 3 h, 6 h…)	`is_synoptic`

Notion	Explication simple	Où dans le code
Épisodes rapides	rafales, fronts, brises : instants difficiles, test de stress	`ep_idx`
Normalisation	(valeur − moyenne) / écart-type, pour mettre les variables à la même échelle	BLOC 5
Masquage	cacher des capteurs au hasard (20 à 100 % visibles) pour apprendre à s'en passer	`KEEP_TRAIN` , `sensor`

Notions : physique du vent

Notion	Explication simple	Où dans le code
Conservation de la masse	l'air ne se crée pas et ne disparaît pas : ce qui entre dans un volume en sort	`mass_residual`
Résidu de masse	quantité d'air « créée ou perdue » en un point ; doit être ≈ 0	`r` , L_mass, DSO2
Condition au sol	au contact du relief, le vent suit la pente : w = u·pente_x + v·pente_y	`w_bc` , L_bc
Vent géostrophique (ug, vg)	vent de grande échelle dû à la différence de pression et à la rotation de la Terre	`geostrophic_from_surface`
Force de Coriolis	effet de la rotation de la Terre qui dévie le vent	`F_CORIOLIS = 8.57e-5`
Vent thermique (su, sv)	changement du vent avec l'altitude, dû aux différences de température	`thermal_wind_from_surface`
Pression réduite au niveau de la mer	pression corrigée de l'altitude du capteur, pour comparer les capteurs entre eux	`p_sl`

Notion	Explication simple	Où dans le code
Moindres carrés	trouver le plan qui passe au plus près de tous les points	`np.linalg.lstsq`
Différences centrées	variation d'une grandeur = (voisin de droite − voisin de gauche) / distance	`grad1d`
Rugosité du champ	mesure de la « finesse » du vent prédit (variations entre voisins)	`roughness`, DSO3
Direction météo	d'où vient le vent (0° = nord, 90° = est)	`direction` (BLOC 12)
Vent calme	vent < 1 m/s : sa direction n'a pas de sens	`calm` (BLOC 10)

Notions : apprentissage et modèle FNO

Notion	Explication simple	Où dans le code
Réseau de neurones	programme qui apprend à transformer une entrée en sortie en ajustant ses poids	`PCFNO3D`
Poids / paramètres	les nombres que le modèle ajuste en apprenant (453 M ici)	`n_par`
FNO (Fourier Neural Operator)	réseau qui travaille sur les « ondes » du champ, ce qui lui donne une vue de tout le domaine d'un coup	`SpectralConv3d`
FFT	transforme une carte en somme d'ondes, et l'inverse	`torch.fft.rfftn` / `irfftn`
Modes	nombre d'ondes gardées : (6, 32, 32) en z, y, x	`MODES`
Lift / projection	25 → 48 canaux au début ; 48 → 128 → 3 (u, v, w) à la fin	`self.lift`, `self.proj`

Notion	Explication simple	Où dans le code
Skip 1 × 1	branche qui garde l'information du point lui-même	`self.skip`
Branche locale 3 × 3 × 3	regarde les voisins proches pour les détails fins	`self.local`, `LOCAL_CONV`
GELU	fonction qui permet d'apprendre des relations non linéaires	`nn.GELU`
Padding	bordure de zéros (4, 8, 8) pour que la FFT ne mélange pas les bords opposés	`PAD`
Loss	note d'erreur à faire baisser : L = L_data + 0,15·L_grad + φ(t)·[L_mass/5e-7 + 0,1·L_bc]	`compute_loss`
L_data	écart entre prédiction et vérité	`l_data`
L_grad	le modèle doit garder les variations fines (pas un vent trop lisse)	`l_grad`, `LAMBDA_GRAD = 0.15`
φ(t), warm-up physique	la physique compte de plus en plus pendant les 10 premières époques	`WARMUP_PHYS_EPOCHS`
Poids de w par niveau	l'erreur de w compte jusqu'à 25 fois plus en altitude, où w est minuscule	`W_LEVEL_W`
Époque	un tour d'entraînement (400 instants) ; 140 au total	`EPOCHS`, `STEPS_PER_EPOCH`
AdamW	méthode qui ajuste les poids pour baisser la loss	`optimizer`
Taux d'apprentissage	taille des corrections ; OneCycle le fait monter puis descendre	`LR = 1e-3`, `OneCycleLR`
Weight decay	petit frein sur les poids contre le surapprentissage	`WEIGHT_DECAY = 1e-4`

Notion	Explication simple	Où dans le code
Surapprentissage	le modèle apprend par cœur et marche mal sur des jours nouveaux	—
Rétropropagation	calcul de la correction de chaque poids à partir de l'erreur	`backward()`
Accumulation	additionne 2 instants avant de corriger les poids	`ACCUM_STEPS = 2`
Clip des gradients	limite les corrections trop brutales	`clip_grad_norm_`
Précision mixte bf16	calculs plus légers donc plus rapides ; la FFT et la sortie restent en fp32	`amp_dtype` , `autocast`
EMA	moyenne des poids dans le temps → modèle plus stable (c'est lui qui est sauvegardé)	`ema_model` , `EMA_DECAY = 0.999`
Checkpoint	fichier de sauvegarde : `best.pt` = meilleur modèle, `last.pt` = pour reprendre	BLOC 9
Workers	processus qui préparent les données pendant que le GPU calcule	`NUM_WORKERS = 4`
GPU	carte graphique qui fait les calculs très vite (A100)	`device`

Notions : évaluation et DSO

Notion	Explication simple	Seuil	Résultat
RMSE	erreur moyenne en m/s (plus petit = mieux)	—	u 0,542 ; v 0,437 (global)

Notion	Explication simple	Seuil	Résultat
R^2	part de la variation du vent bien reproduite (1 = parfait)	—	u, v 0,991–0,998 ; w ⩾ 0,966
DSO1 : précision	RMSE et R^2 de u, v, w à chaque niveau	RMSE ⩽ 0,5 m/s ; R^2 ⩾ 0,95	R^2 atteint à tous les niveaux
DSO2 : physique	résidu de masse moyen (RMS)	⩽ 1e-4	2,58e-5
DSO3 : finesse	rugosité prédite / rugosité vraie (pas de bruit artificiel)	⩽ 1,1	0,951
DSO4 : vitesse	temps de calcul d'un instant	< 1 s	91 ms
DSO5 : robustesse	RMSE avec seulement 25 % des capteurs	⩽ 0,7 m/s	0,646 m/s
MAE direction	erreur moyenne de direction (vents ⩾ 1 m/s)	—	3,6°
Couverture d'un niveau	part du niveau au-dessus du relief ; < 5 % = niveau exclu	`MIN_LEVEL_COVER = 0.05`	—
Robustesse	notes avec 10, 25, 50 et 100 % des capteurs visibles	—	`robustesse_capteurs.csv`
TTA (moyenne de prédictions)	prédire plusieurs fois avec des capteurs différents, puis moyenner	< 1 s	testé au BLOC 10c
Fine-tuning (BLOC 9b)	petit réentraînement orienté DSO	—	testé mais **non retenu** (dégradait les résultats)

Ce que fait chaque bloc du notebook

Les blocs s'exécutent de haut en bas ; le modèle final vient du BLOC 9 (le 9b n'a pas été utilisé).

Bloc	Ce qu'il fait	Ce qu'il produit
0 — Préparation	charge les bibliothèques, monte Google Drive, choisit le GPU, fixe la graine du hasard (42)	`device`, outils prêts
1 — Configuration	regroupe tous les réglages : dossiers, grille, taille du modèle, entraînement, poids de la loss, seuils DSO	`NZ`, `WIDTH`, `EPOCHS`, `LAMBDA_…`, `DSO…`
2 — Grille et terrain	lit le relief, calcule les points valides, les masques, les pentes, les 9 canaux fixes, le pooling 2 × 2 et les opérateurs physiques	`VALID`, `STATIC_2D/3D`, `mass_residual`, `C_IN = 25`
3 — Cache	convertit les CSV de chaque jour en un fichier surface + un fichier vérité par instant	`_S.npy`, `_Y.npy`, `INDEX`
4 — Contrôle	vérifie que la vérité respecte la conservation de la masse et la condition au sol	arrêt si les données sont incohérentes
5 — Normalisation	calcule moyennes et écarts-types (jours d'entraînement seulement) et le poids de w par niveau	`norm_stats.json`, `SURF_MU/SD`, `Y_MU/SD`, `W_LEVEL_W`
6 — Données	calcule vents géostrophique et thermique, masque des capteurs au hasard, assemble les 25 canaux	`train_loader`, `val_loader`, `test_loader`, `assemble_input`
7 — Modèle	définit le PC-FNO3D : lift, 4 couches (Fourier + skip + local), projection	`model` (453 M paramètres)
8 — Loss	définit la note d'erreur (données + gradients + masse + sol), le R^2 et la rugosité	`compute_loss`, `roughness`

Bloc	Ce qu'il fait	Ce qu'il produit
9 — Entraînement	140 époques avec AdamW, OneCycle, bf16, EMA ; valide à chaque époque ; sauvegarde et reprise	`best.pt`, `last.pt`, `history.json`
9b — Raffinement	20 époques de fine-tuning orienté DSO	testé, **non retenu**
10 — Évaluation	note le modèle sur les jours de test : RMSE et R^2 par niveau, masse, rugosité, temps, robustesse, direction ; verdict pour chaque DSO	CSV de résultats, `test_summary.json`
10c — TTA	compare 1 prédiction, 8 prédictions moyennées, 2 modèles × 8	tableau comparatif + temps
11 — Figures	trace les courbes d'apprentissage, les notes par niveau, les cartes vérité/prédiction/erreur, un profil vertical, la robustesse	5 images PNG
12 — Reconstruction	reconstruit le vent 3D d'un instant (cache ou nouveau CSV), l'enregistre et trace une carte à 120 m + une coupe verticale	`.npz`, `.csv`, figure

Le chemin en une ligne : CSV → cache (3) → normalisation (5) → 25 canaux (6) → modèle (7) entraîné avec la loss physique (8, 9) → évaluation DSO (10) → vent 3D livré (12).